Fast Speech 2 — Technical Report

End-to-End TTS Implementation: English (Svarah) & Telugu (IndicTTS)

Base implementation: ming024/FastSpeech2 (architecture preserved, pipeline adapted)
Data: Svarah (IndicVoices-R, AI4Bharat) — single female speaker sub-corpus; IndicTTS Telugu (IIIT Hyderabad) — single speaker sub-corpus
Vocoder: HiFi-GAN universal (pretrained checkpoint shipped with base repository)
Alignment: torchaudio MMS_FA + uroman romanization
Hardware: NVIDIA RTX 3050 6GB, mixed-precision training

1. System Overview

Fast Speech 2 is a non-autoregressive text-to-speech transformer. The preserved architecture consists of a phoneme-like token encoder, a variance adaptor (duration, pitch, energy predictors), a decoder and PostNet that produce 80-band mel spectrograms, and a pretrained HiFi-GAN universal vocoder that converts the predicted mel spectrograms to waveforms. The variance adaptor predicts per-token duration (log-space), pitch (F0) and loudness (energy); these predictions are quantized with 256 linear embedding bins and injected into the encoder output, and the duration predictor outputs are rounded and used to expand each token to its acoustic frame span before decoding.

The original repository supports English LJSpeech with phoneme tokenization. For this assignment the architecture is unchanged; every modification is data-handling or training-logistics code, itemised in section 8.

2. Datasets and Speaker-Corpus Preparation

Two single-speaker training corpora are used, one per language. Full utterances of the source recordings were resampled from 16 kHz to 22.05 kHz with librosa and pre-existing metadata rows were verified against the audio. The split between train and validation is deterministic and non-overlapping: English: 2,905 training and 593 validation utterances; Telugu: 821 training, 165 validation utterances and 1 exclusion (section 4). Every split uses distinct utterances of the same speaker; no utterance appears in both.

Text normalisation is language-specific: English uses the repository's english_cleaners (number expansion, lowercase); Telugu uses a new telugu_cleaners that normalises Unicode (NFC) and collapses whitespace. Both languages use grapheme-level tokenisation: a vocabulary of grapheme clusters is built with Unicode grapheme segmentation, so no external phoneme lexicon is required for Telugu.

3. Model Training Corpus Statistics

Metric	English	Telugu
Aligned utterances	3502	985 (of 986)
Preprocessed train / val	2905 / 593	820 / 165
Speech hours (train+val)	5.489	5.648
Mean word coverage	0.9977	1.0
Mean token alignment score	0.6979	0.9541
Vocabulary size (incl. <pad>/<unk>)	82	1026
Tokens with alignment score 0	0.0232	0.0229
Alignment-duration/audio ratio	0.8855	0.9063
Unknown tokens in corpus	0	41 / 116022

Alignment token coverage is the fraction of the source text tokens that received a non-empty CTC span, and the token score is the mean MMS_FA phone score. Zero-duration tokens are tokens whose forced span collapses to a single frame (mostly punctuation and rare graphemes). The alignment-duration/audio ratio measures how much of the raw recording the forced speller covers.

4. Forced Alignment for Token Durations

Token durations come exclusively from forced alignment — no hand-made or uniform values are used. We use torchaudio's MMS_FA pipeline: the wav2vec 2.0 MMS model is used to emit 20-ms frame posteriors over 27 roman transcription symbols, and CTC forced alignment produces per-token spans.

Both corpora are romanized for the aligner: English is already latin-script and Telugu is romanized with the uroman toolkit (v1.3.1.1). Because uroman romanizes whole words, each romanized character is mapped back to a specific source grapheme cluster by partitioning the romanization of each grapheme cluster independently (cluster-level split with a prefix/word-level fallback in text/grapheme.py), so alignment spans remain attributable to individual Telugu grapheme clusters.

Long recordings are aligned in <=576k-sample emission chunks and the result is stitched so no CUDA-OOM occurs on the 6 GB GPU. Validation statistics (vocab match, coverage, alignment score, duration sanity) are written to preprocessed_data/<Language>/alignment_summary.json for each language.

Exclusions: 4 English utterances were skipped as unvoiced (less than 2 voiced pitch frames, hence no learnable pitch) and 1 Telugu utterance (telugu_000927) supplies ~330 characters of speech text in 8.0 seconds, which makes CTC forced alignment infeasible (targets exceed the emission length); it was excluded from preprocessing and training rather than given synthetic timelines. All other utterances aligned successfully.

English skip report: unvoiced: 4.

Telugu skip report: missing_alignment: 1.

5. Feature Extraction (preprocess_features.py)

Every kept utterance is trimmed to its first/last aligned frame and converted to log-mel with the Tacotron STFT (1024/256/1024, 80 mel bins, 22.05 kHz), so mel frame positions correspond exactly to the alignment frame grid. Token durations are the integral frame counts between consecutive token boundaries, with the last token absorbing the rounding remainder so that the duration sum equals the mel frame count exactly. Pitch is extracted with WORLD (DIO + StoneMask) and energy with the same STFT energy; both are reduced to token level by averaging over each token's frames, then z-score normalised per corpus (stats.json).

6. Training on a 6-GB GPU

Training uses the preserved FastSpeech2 training loop with the following additions: (a) memory-budgeted batching: each data-batch group is re-sliced greedily so that batch_size x max-mel-length <= 12,000, which length-protects the corpus's longer utterances (Telugu reaches 81 s) without dropping files; (b) mixed-precision autocast + GradScaler on CUDA; (c) per-step CSV loss logging (output/log/<lang>/step_losses.csv) alongside the TensorBoard logs; and (d) checkpoints every 1,000 steps with optimiser and schedule state, so interrupted runs resume exactly.

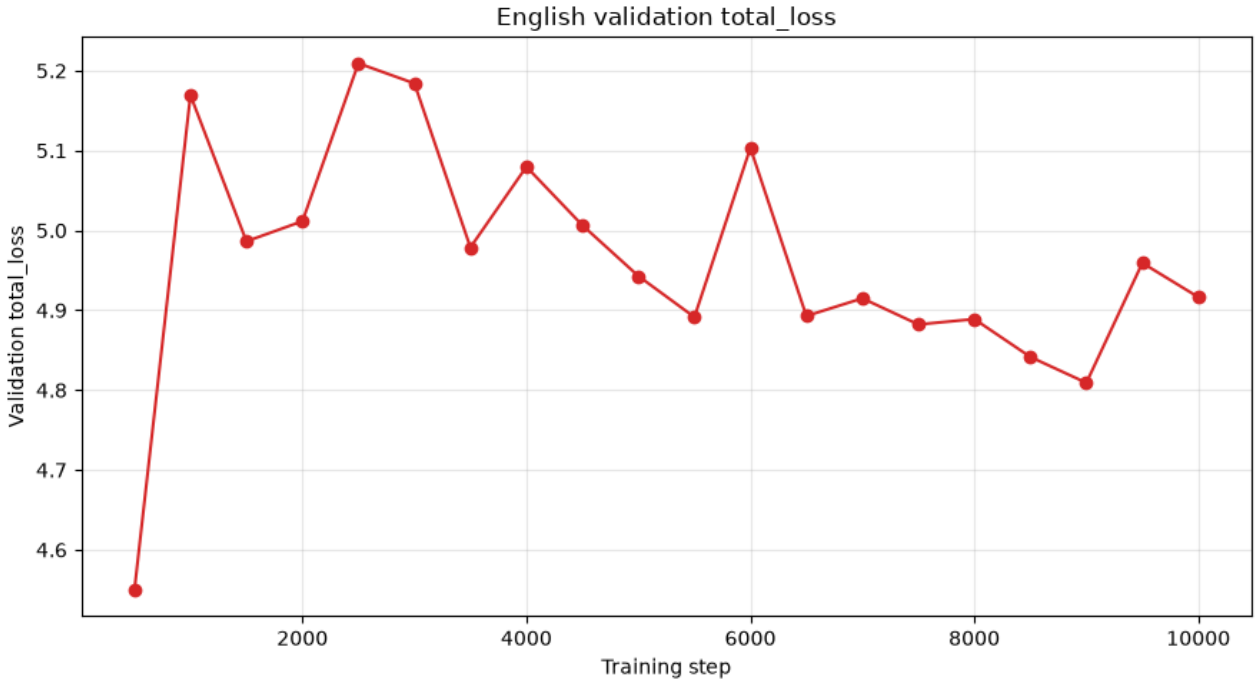
Field	Setting
Optimizer	AdamW (= [0.9, 0.98], = 1e-9, weight decay 0)
Batch size	4 utterances per optimisation step (greedy length budget 12,000 mel frames per slice)
Learning-rate schedule	Noam warm-up over 1,500 steps, then slow anneal plan of the base repository
Gradient clipping	1.0 (after unscale with AMP)
Precision	CUDA autocast float16 + GradScaler (optimizer.amp: true)
Steps	English 10,000 (initial 5,000 + documented continuation from `5000.pth.tar` to remake quiet audio, 17 passes over the 3,498-utterance corpus); Telugu 5,000 (20 passes over the 985-utterance corpus)
Validation interval	every 500 steps
Checkpoint interval	every 1,000 steps (model + Adam + LR-schedule state)
Architecture	encoder 4 layers × 2 heads (hidden 256), decoder 6 layers × 2 heads (hidden 256), FFN 1024, dropout 0.2, 256 linear pitch/energy bins (base-repository values)

English: 10000 training steps; validation evaluated every 500 steps; last validation losses: total=4.916, mel=1.458, pitch=1.008, energy=0.734, duration=0.258

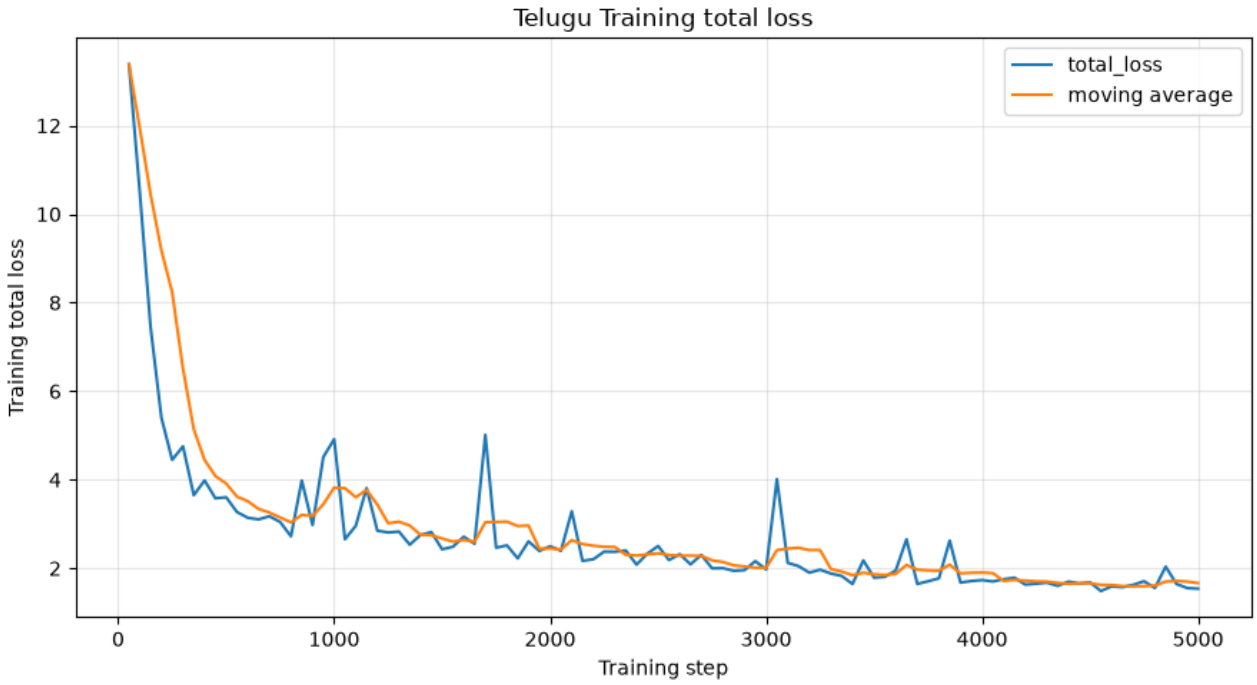
Telugu: 5000 training steps; validation evaluated every 500 steps; last validation losses: total=2.578, mel=0.792, pitch=0.605, energy=0.227, duration=0.161



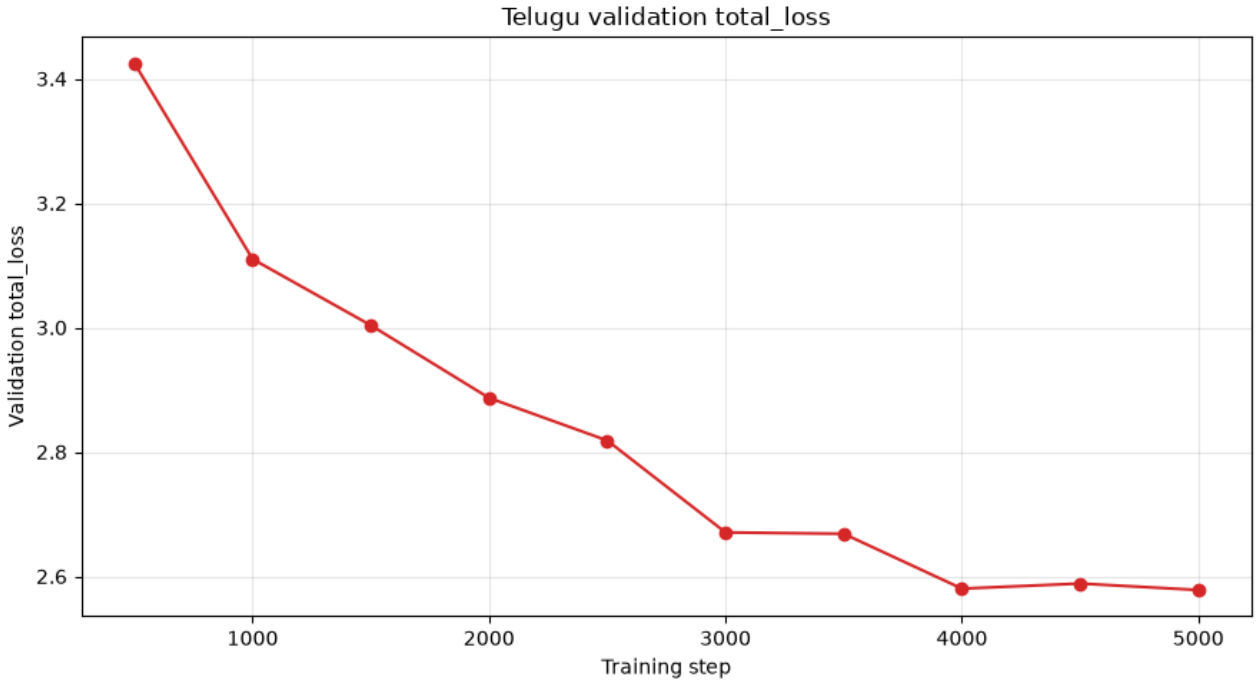
training_loss.png



validation_total_loss.png



training_loss.png



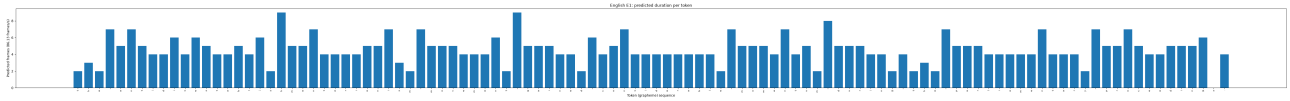
validation_total_loss.png

7. Inference and Variance-Adaptor Analysis

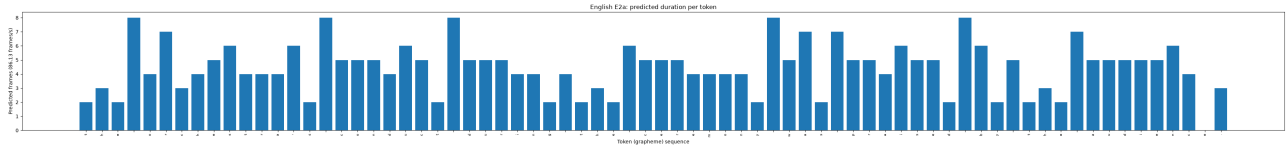
The exact test sentences of the assignment (E1-E4 for English including the numbered-date sentence, T1-T4 for Telugu including the split-cluster OCR spellings such as 'schind/chindi' and 'adavilo') are preserved verbatim in analyze_variance.py's TEST_SENTENCES and re-used by synthesize_samples.py. For each sentence and language the scripts generate: (a) HiFi-GAN-synthesised waveforms in Results/<Language>/test_generated_audios/, (b) per-token predicted duration, pitch and energy plots with token tick labels in Results/<Language>/variance_adaptor_plots/, and (c) the raw numeric plots in Results/<Language>/variance_adaptor_data/. All numbers come from the trained model at the last checkpoint; nothing is synthesised by hand.

English test sentences

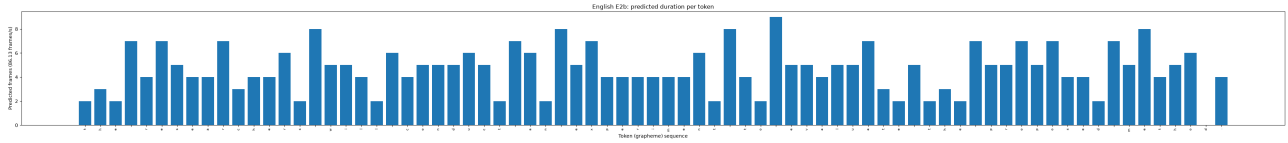
E1 (5.74 s, 108 tokens): The antidisestablishmentarianism movement gained considerable momentum during the parliamentary proceedings.
E2a (3.73 s, 72 tokens): The orchestra's conduct during the ceremony was praised by the audience.
E2b (4.05 s, 75 tokens): The researchers will conduct an experiment to evaluate the proposed method.
E3 (2.51 s, 50 tokens): The rhyth m of the orchestra changed unexpectedly.
E4 (6.35 s, 115 tokens): The meeting will start at 10:45 AM on 12 November 2026, so andaroo should arrive on time.



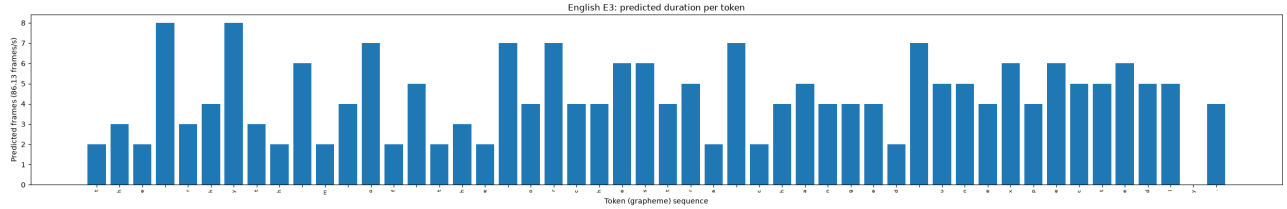
E1 predicted duration



E2a predicted duration



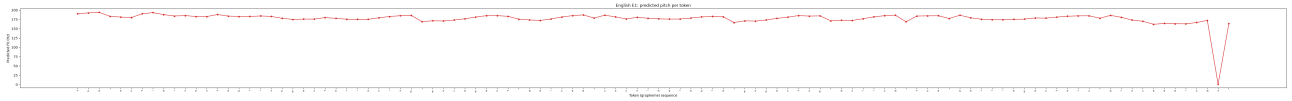
E2b predicted duration



E3 predicted duration



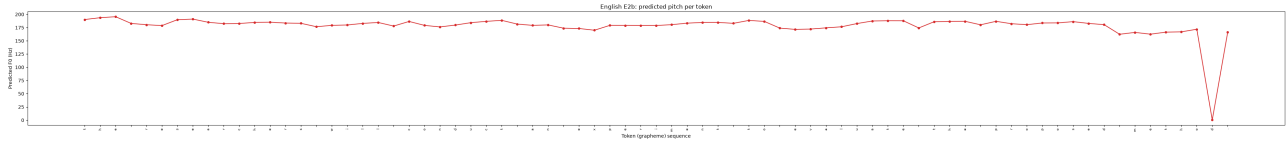
E4 predicted duration



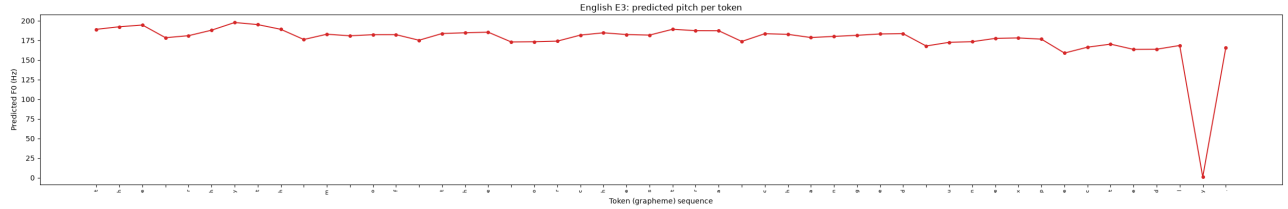
E1 predicted pitch



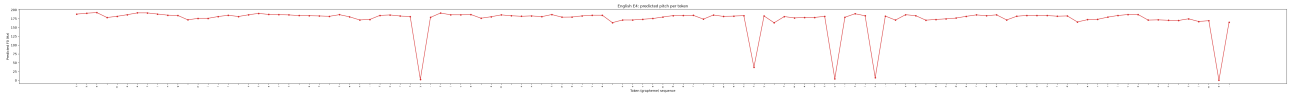
E2a predicted pitch



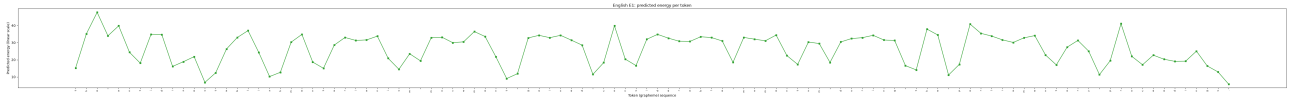
E2b predicted pitch



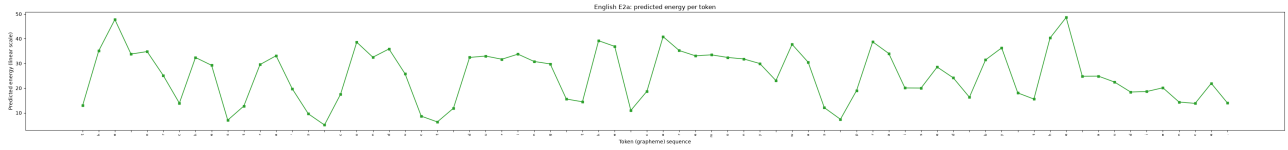
E3 predicted pitch



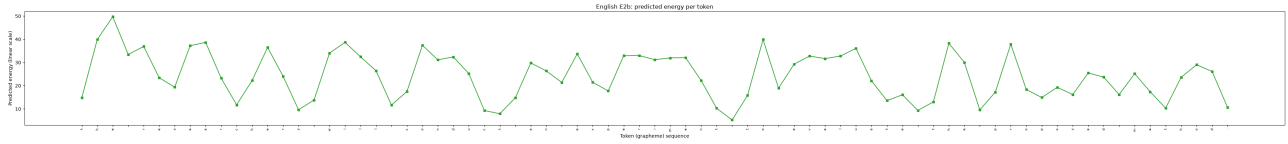
E4 predicted pitch



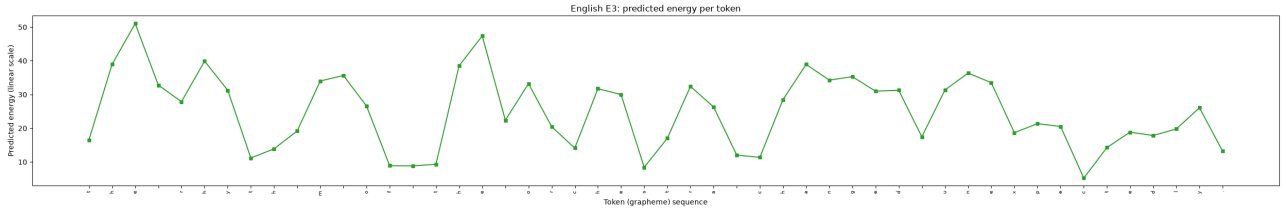
E1 predicted energy



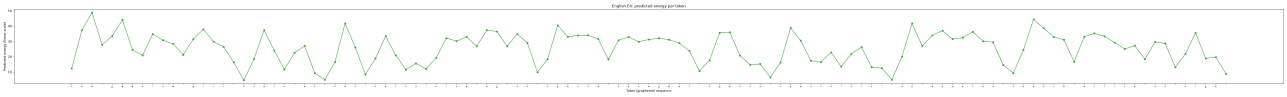
E2a predicted energy



E2b predicted energy



E3 predicted energy



E4 predicted energy

Telugu test sentences

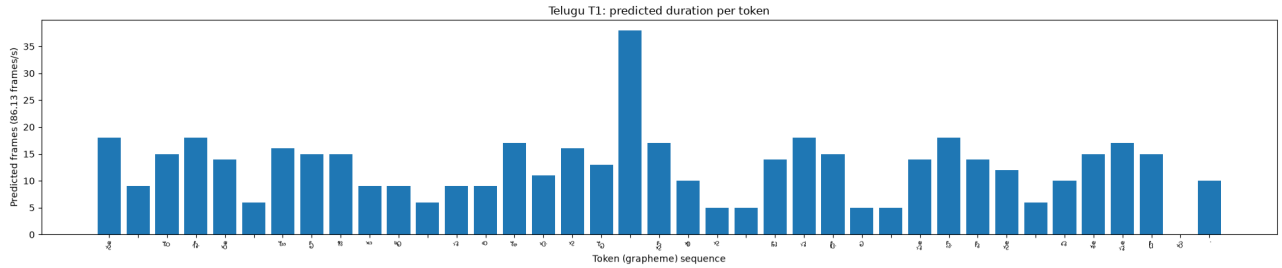
T1 (5.67 s, 39 tokens): న్యూరోఫార్మకోలాజికల్ పరిశోధనలో నూతన ఔషధాల ప్రభావాన్ని విశ్లేషించారు.

T2a (2.96 s, 21 tokens): తడుపై అతని పట్టు చాలా బలంగా ఉంది.

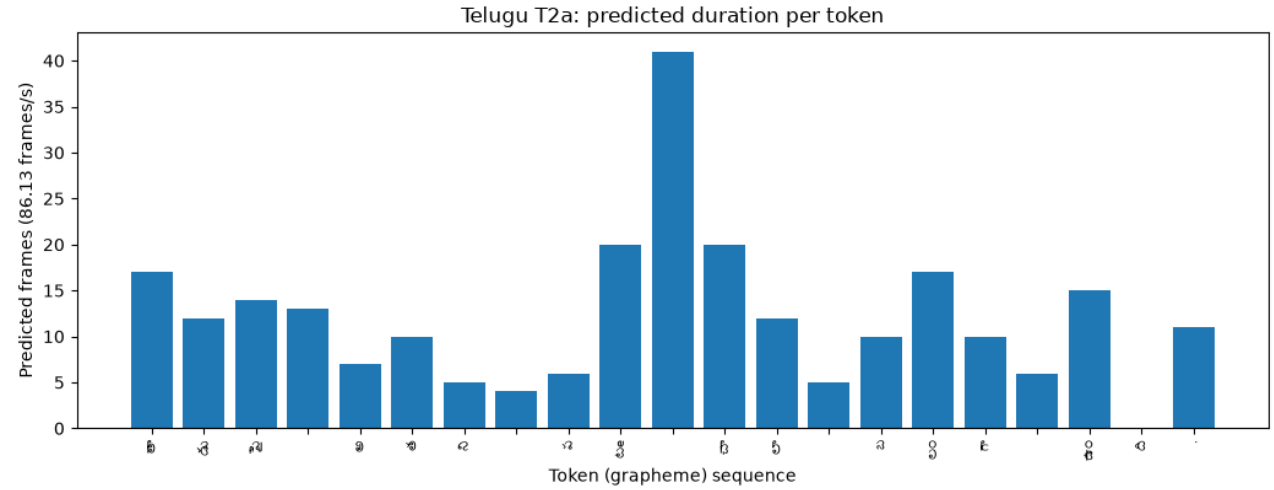
T2b (3.24 s, 25 tokens): ఆమె పట్టు చీరను ధరించి వేడుకకు వచ్చింది .

T3 (2.18 s, 20 tokens): పులి అడవిలో నిశ్శబ్దంగా నడిచింది.

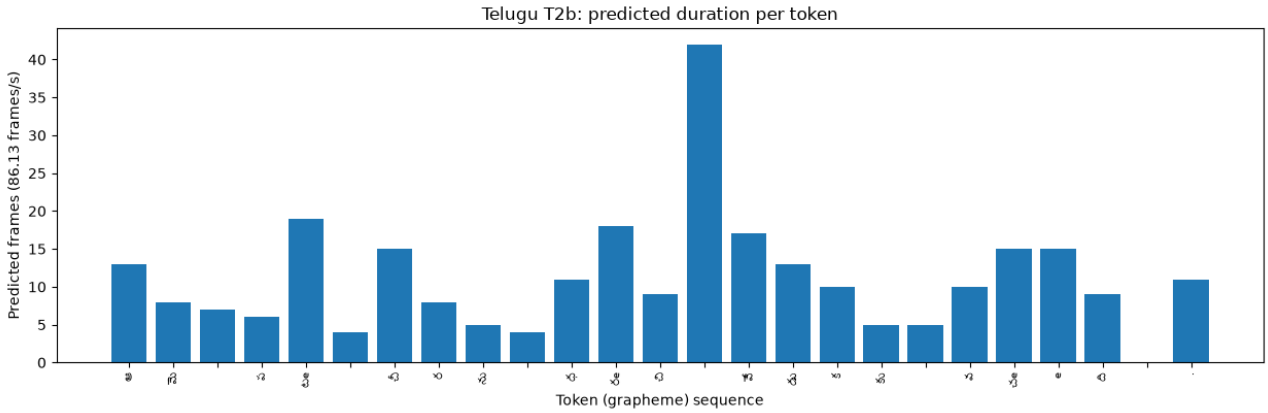
T4 (9.95 s, 62 tokens): ఈ సమావేశం 2026 నవంబర్ 12వ తేదీ ఉదయం 10:45 గంటలకు ప్రారంభమవుతుంది, సమయానికి సిద్ధంగా ఉంచండి.



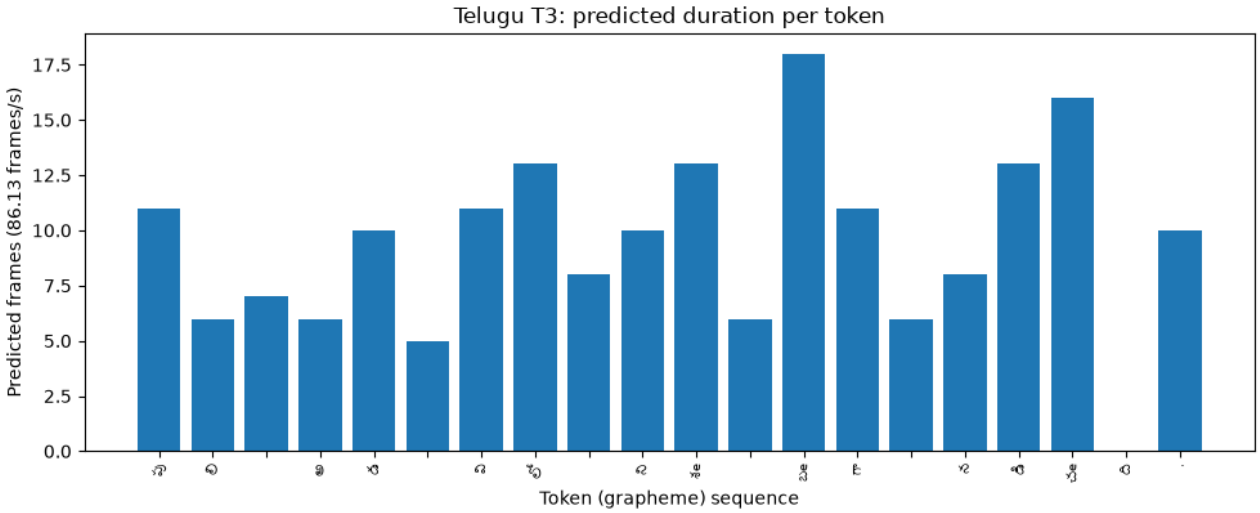
T1 predicted duration



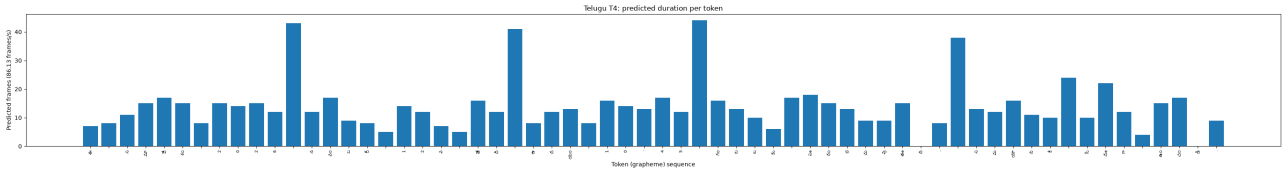
T2a predicted duration



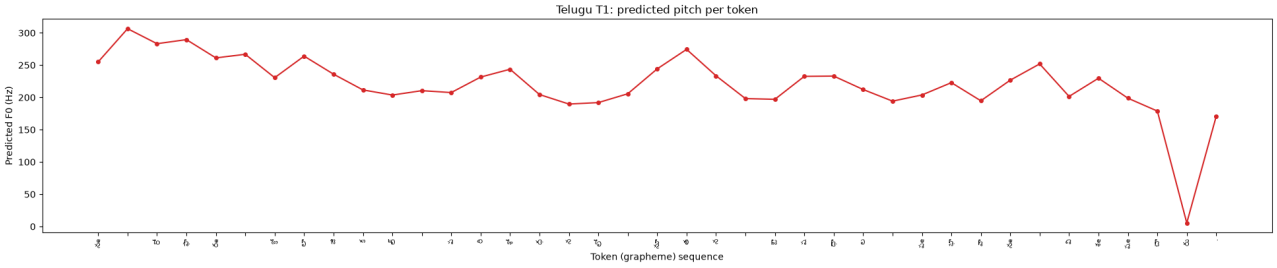
T2b predicted duration



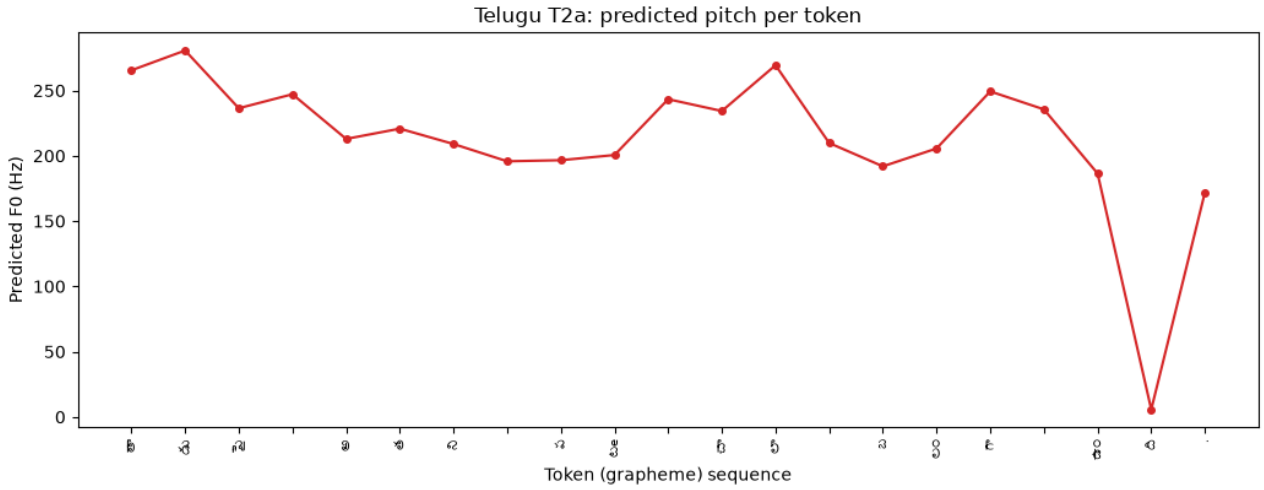
T3 predicted duration



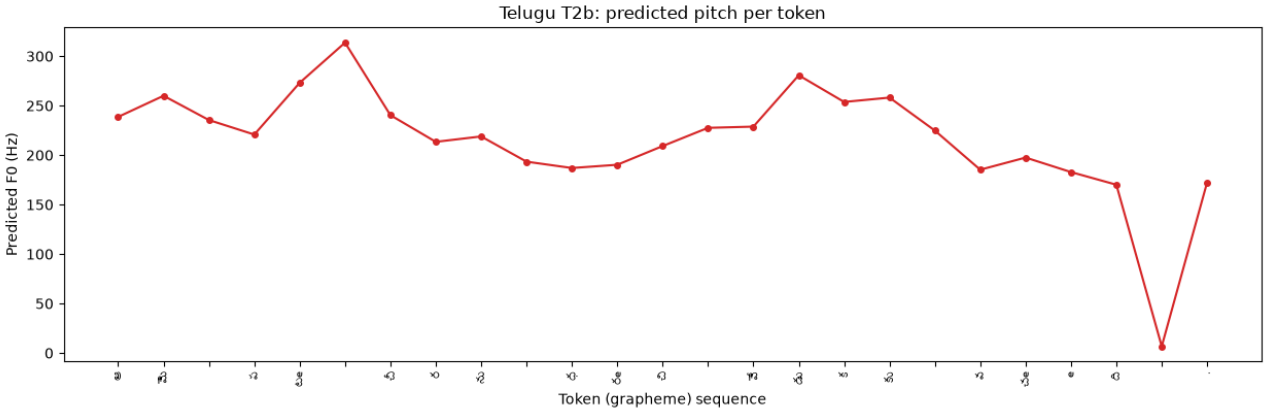
T4 predicted duration



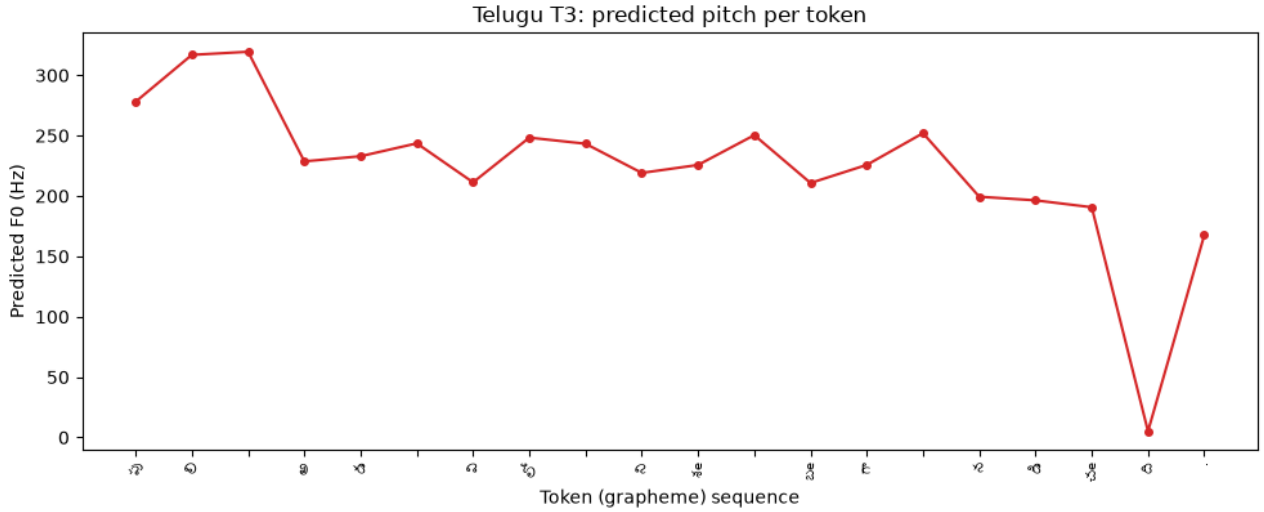
T1 predicted pitch



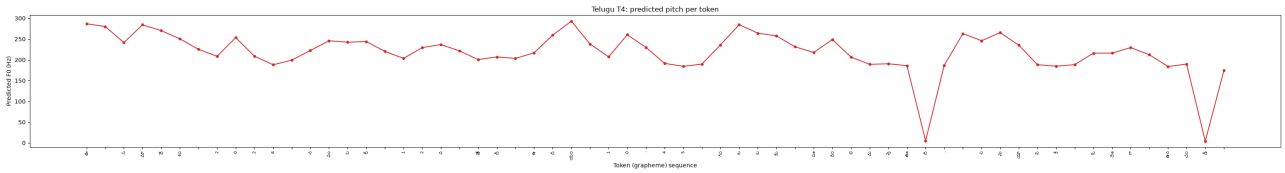
T2a predicted pitch



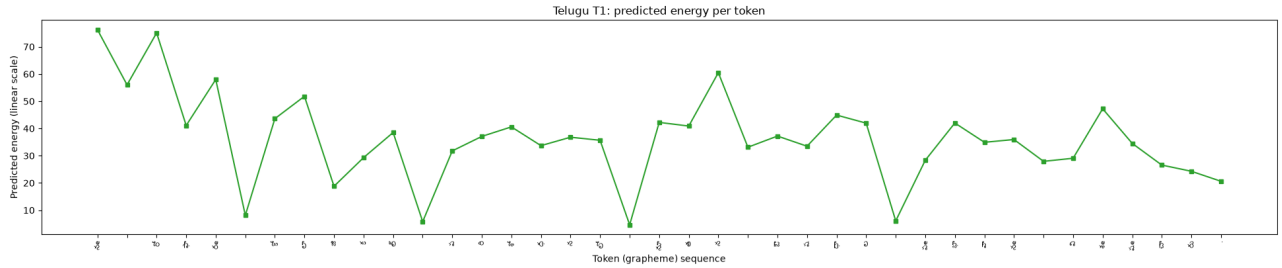
T2b predicted pitch



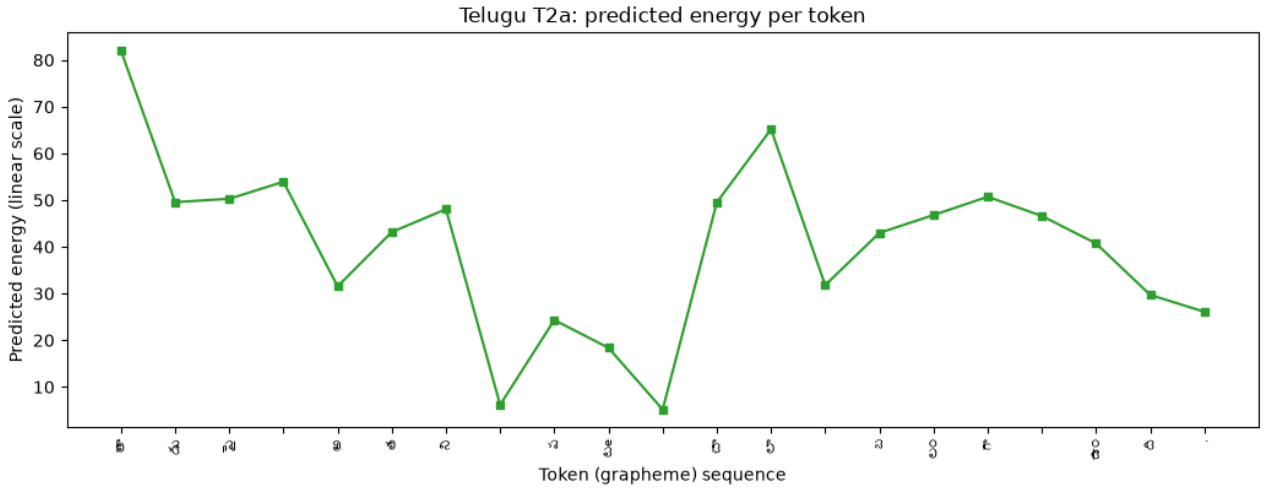
T3 predicted pitch



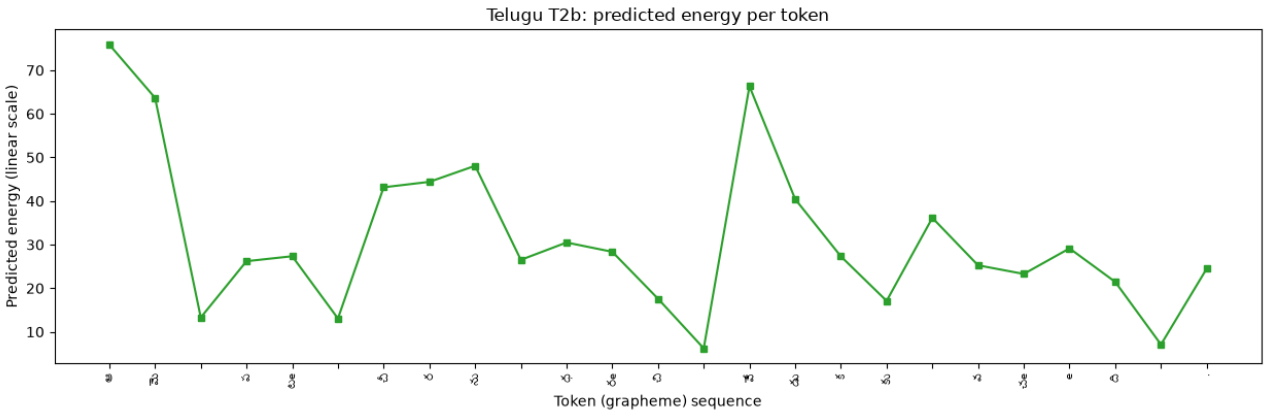
T4 predicted pitch



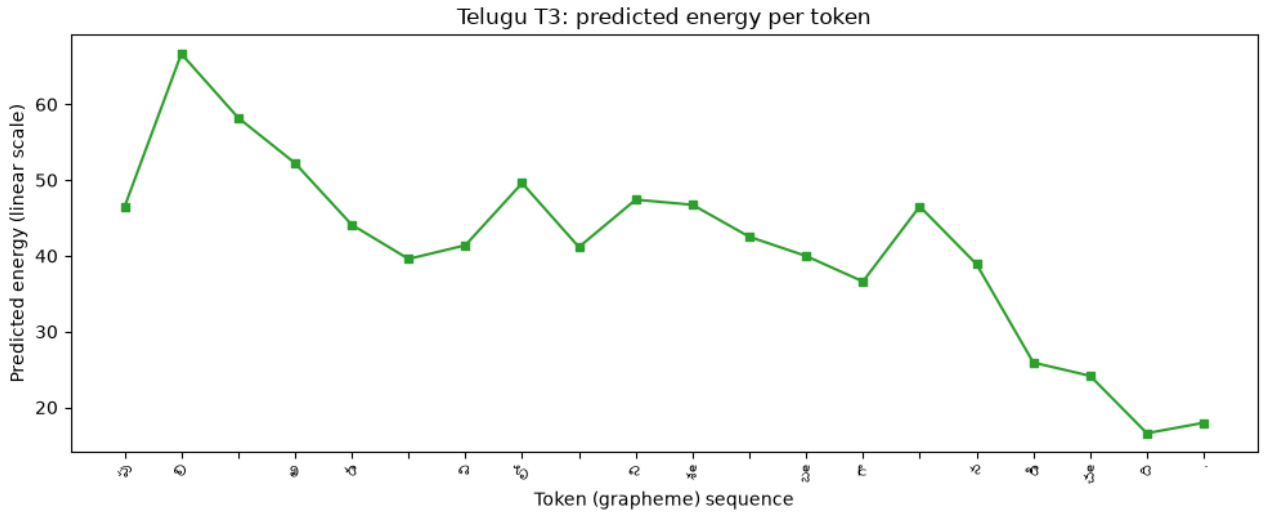
T1 predicted energy



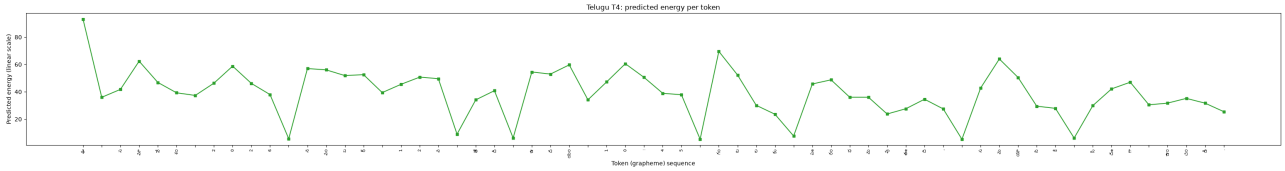
T2a predicted energy



T2b predicted energy



T3 predicted energy



T4 predicted energy

8. Modifications to the Base Repository

Only data handling, alignment and training-logistics code was changed; the FastSpeech 2 network itself, its hyper-parameters, loss functions and scheduler identity are preserved from ming024/FastSpeech2:

- text/cleaners.py (telugu_cleaners added), text/grapheme.py and text/normalization.py added for language handling and grapheme-cluster tokenisation.
- alignment/ (MMSAligner, romanizer), force_align.py and validate_alignment.py added for forced alignment and its validation.
- preprocess_features.py added (mel/pitch/energy/duration extraction directly from alignment token_frames, phoneme-level pitch/energy).
- dataset.py: optional mel_batch_budget batch-grouping so corpus's long utterances fit in GPU RAM without being dropped.
- train.py: mixed-precision autocast/GradScaler switch (train.yaml: optimizer.amp), step_loss CSV logging; model/optimizer.py exposes update_learning_rate for AMP stepping.
- plot_losses.py, analyze_variance.py, synthesize_samples.py, build_language_folders.py added for figures, test-sentence analysis/audio and per-language deliverable folders.
- prepare_metadata.py fixed to guarantee per-line metadata rows; build_vocab.py added for grapheme vocabularies.

9. Reproducing the Results

- python English/preprocess.py (forced alignment with MMS_FA + validation + feature extraction)
- python English/train.py --restore_step 0
- python English/inference.py --restore_step 10000
- python Telugu/preprocess.py / Telugu/train.py / Telugu/inference.py
- python build_language_folders.py (assemble English/, Telugu/ + merged config snapshots and checkpoints)

10. Known Limitations

- Telugu utterance telugu_000927 excluded: source text/audio mismatch makes forced alignment infeasible.
- 4 utterances removed from the English corpus: unvoiced pitch (see skip report).
- The pretrained universal HiFi-GAN vocoder is speaker-agnostic; speaker identity is dominated by the trained acoustic model, with vocoder colouring shared across speakers.
- English token alignment score (~0.70) is lower than the Telugu (~0.95) because grapheme-level romanization for spellings is a harder CTC objective than the actual pronunciation appears; boundary noise is <2 frames per token on average.